October 9, 2026

Agenda (1/2)

- Bluetooth technology: origin, operating bands and power classes
- Bluetooth Low Energy: motivation and comparison with classic Bluetooth
- Physical layer and Link Layer
- Advertising, scanning and connection establishment
- Generic Access Profile (GAP)
- Attribute Protocol (ATT) and Generic Attribute Profile (GATT)

Agenda (2/2)

- Security: pairing, authentication and bonding
- Applications and management tools
- Version history, privacy and hardware-free alternatives (emulation and simulation)

Bluetooth: Background (1/2)

- A short-range wireless technology standard, intended for data exchange between fixed and mobile devices over short distances.
- It operates by radio waves in the ISM bands, between 2.402 GHz and 2.48 GHz.
- It supports the creation of personal area networks (PAN).
- It was originally conceived as a wireless alternative to RS-232 data cables.
- Early applications included file exchange between nearby portable devices and connecting mobile phones and music players to wireless headsets.

Bluetooth: Background (2/2)

- At 2.5 mW of power, typical range is limited to about 10 metres.
- The standard is managed by the Bluetooth Special Interest Group (Bluetooth SIG).

Classic Bluetooth (BR/EDR): Radio and Network (1/3)

- **Frequency hopping (FHSS):** 79 channels of 1 MHz in the 2.4 GHz band, about 1600 hops per second, with 625 µs slots. *Adaptive Frequency Hopping* (AFH, v1.2) avoids channels used by Wi-Fi.
- **Air data rate:** 1 Mbit/s in BR (GFSK); 2 and 3 Mbit/s in EDR ($\pi/4$-DQPSK and 8DPSK modulation).
- **Piconet:** one master and up to 7 active slaves (3-bit address); a *scatternet* joins several piconets through a shared device.
- **Links:** ACL (data, packet switched) and SCO/eSCO (voice, periodic slot reservation); without them there is no low-latency audio.
- **Discovery:** *inquiry* (who is nearby?) followed by *page* (connect to a known address), both much heavier than BLE advertising.

Classic Bluetooth: Power Classes and Profiles (2/3)

Class	Max. power	Typical range	Example
1	100 mW (+20 dBm)	$\approx$ 100 m	USB adapters, industrial
2	2.5 mW (+4 dBm)	$\approx$ 10 m	phones, headsets
3	1 mW (0 dBm)	$\approx$ 1 m	short-range peripherals

- **Profiles** (standardised use cases): SPP (serial port over RFCOMM), HFP/HSP (hands-free), A2DP/AVRCP (audio and control), HID (keyboard, mouse), PAN (IP networking).
- **Link to the IoT lab:** cheap modules such as the HC-05/HC-06 use SPP, the "wireless RS-232 cable" of Bluetooth's origins; they do not show up in a BLE `lescan`.

Evolution of the Bluetooth Versions (3/3)

Version	Year	Main novelty
1.x / 2.0+EDR	1999–2004	BR; EDR (2 and 3 Mbit/s)
2.1	2007	Secure Simple Pairing (SSP)
4.0	2010	**Bluetooth Low Energy**; dual mode
4.1 / 4.2	2013–14	LE Secure Connections (ECDH), 251 B packets, IP over BLE (IPSP)
5.0	2016	2M and Coded PHY, extended advertising (255 B), 8× broadcast
5.1	2019	Direction finding (AoA/AoD)
5.2	2019	LE Audio (LC3), isochronous channels, EATT
5.3 / 5.4	2021–23	Efficiency improvements; PAwR and electronic shelf labels
6.0	2024	*Channel Sounding* (distance measurement)

Bluetooth Low Energy: Motivation (1/2)

- A wireless networking standard developed by the Bluetooth Special Interest Group.
- It constitutes a lightweight subset of classic Bluetooth, and not merely an optimisation of it.
- It was added to the standard in version 4.0 of the Bluetooth Core Specification, adopted in 2010.
- It was designed for low-power, low-latency, low-throughput wireless applications.

Bluetooth Low Energy: Motivation (2/2)

- It is not backward compatible with classic Bluetooth devices: a device that needs to communicate with both must implement both stacks.
- Hence the distinction between *single mode* devices, implementing only BLE, and *dual mode* devices, supporting both BLE and classic Bluetooth.

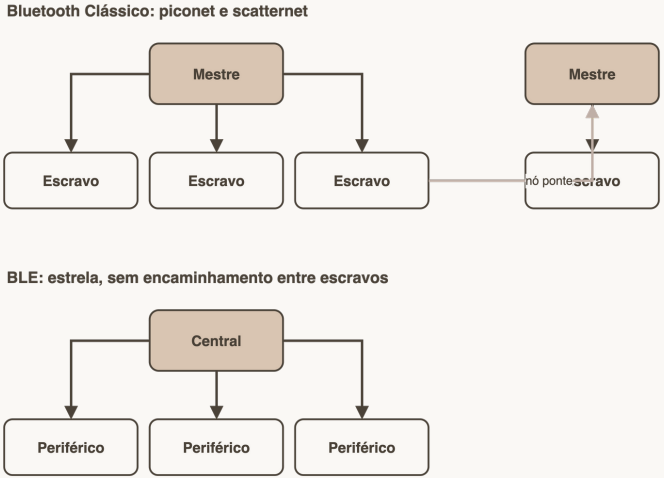
BLE Compared with Classic Bluetooth (1/2)

- **Topology:** BLE is organised as a star; classic Bluetooth supports *scatternet*.
- **Application throughput:** about 0.27 Mbit/s in BLE, against 0.7 to 2.1 Mbit/s in classic Bluetooth; over the air, 1 Mbit/s against 1 to 3 Mbit/s.
- **Message size:** 8 to 47 bytes in BLE, against 358 bytes in classic Bluetooth.
- **Consumption:** below 15 mA in BLE, against under 30 mA in classic Bluetooth.
- **Cost:** low in BLE, high in classic Bluetooth.
- **Range:** 50 to 150 m in BLE, under 30 m in classic Bluetooth.

BLE Compared with Classic Bluetooth (2/2)

- **Latency:** 3 ms in BLE, against about 100 ms in classic Bluetooth.
- Reference consumption for classic Bluetooth is around 1 W, whereas BLE sits between 0.01 and 0.5 W depending on the use case.

Topologies: Classic and BLE



Device Roles in BLE (1/2)

- **Peripheral device (server):** small, low-power and resource-constrained, it connects to a more capable central device. Examples include heart rate monitors and BLE proximity tags.
- **Central device (client):** it has greater processing and memory capacity. Examples include mobile phones and tablets.

Device Roles in BLE (2/2)

- This asymmetry is deliberate: it concentrates complexity in the device that has energy available and keeps the sensor simple and inexpensive.
- Three distinct pairs of roles should not be confused, as they operate at different layers: master and slave, at the link layer; central and peripheral, at GAP level; client and server, at GATT level.

BLE Physical Layer (1/2)

- It operates in the 2.4 GHz ISM band.
- Data rate of 1 Mbit/s, with GFSK modulation.
- The modulation index is higher than that of basic rate Bluetooth, which translates into better range.
- The spectrum is divided into 40 channels, with 2 MHz spacing.

BLE Physical Layer (2/2)

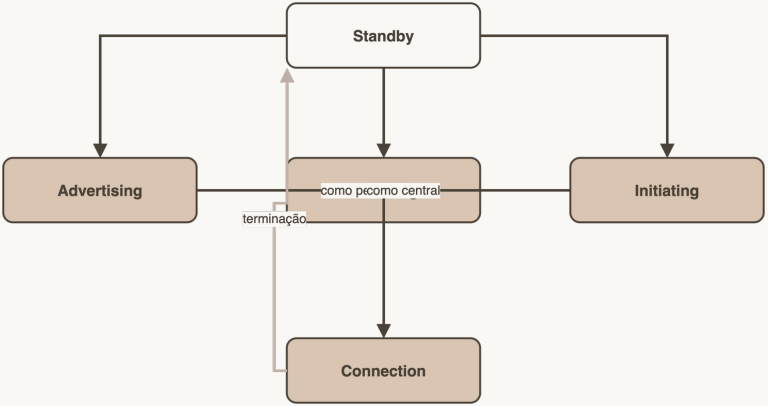
- Three of those channels (37, 38 and 39) are reserved for advertising; the remaining ones, numbered 0 to 36, carry data during established connections.
- Separating advertising channels from data channels ensures that device discovery does not interfere with connections already in progress.

Bluetooth 5: New Physical Layers

PHY	Rate	Relative range	Note
LE 1M	1 Mbit/s	$1\times$	mandatory; BLE 4.x
LE 2M	2 Mbit/s	$\approx 1\times$ (smaller margin)	half the airtime per bit
LE Coded S=2	500 kbit/s	$\approx 2\times$	FEC coding
LE Coded S=8	125 kbit/s	$\approx 4\times$	maximum range, lowest rate

- **Extended advertising** uses the data channels as secondary channels and carries up to 255 bytes per packet (against 31 bytes in legacy advertising).
- Trading rate for range is the same trade-off as in the LPWAN networks of Lesson 03, now at tens to hundreds of metres.
- A phone or adapter without 5.0 support does not see extended advertisements: check the version with `hciconfig -a`.

Link Layer State Machine



Link Layer: States (1/2)

- **Standby:** the device neither transmits nor receives data, and is not connected to any other device.
- **Advertiser:** it periodically broadcasts advertising packets.
- **Scanner:** it actively looks for devices in the advertising state.
- **Initiator:** it actively attempts to initiate a connection with another device.
- **Master:** it is connected to another device, in the master role.

Link Layer: States (2/2)

- **Slave:** it is connected to another device, in the slave role.
- The transition between these six states describes the entire life cycle of a BLE communication, from discovery to connection termination.

Advertising Events (1/2)

- During an advertising event, one packet is transmitted on each of the three advertising channels: 37, 38 and 39.
- Repetition across the three channels increases the probability of the advertisement being received, even if one of the channels is subject to interference.
- A device in the advertising state transmits packets that may carry a data payload.

Advertising Events (2/2)

- Advertisements may be directed at a specific device or undirected, that is, intended for any receiver.
- Advertisements may further be connectable or non-connectable; in the latter case they serve only to broadcast data, with no intention of establishing a connection.

Advertising Types and Scanning (1/2)

- **Connectable undirected:** any scanning device may initiate a connection with this advertiser.
- **Connectable directed:** only one specific device may initiate the connection.
- **Non-connectable undirected:** no device may initiate a connection; used mainly for general data broadcasting.
- **Discoverable undirected:** any scanning device may request further information from the advertiser, but may not establish a connection.

Advertising Types and Scanning (2/2)

- **Passive scanning:** the device listens to the advertising channels and, on receiving a packet, forwards the information to the upper layers.
- **Active scanning:** the device responds to the advertisement with a scan request, to which the advertiser replies with additional information.

Advertising Packet Structure (1/2)

- The legacy payload has up to **31 bytes**, organised in *AD* structures: length, type and value. The scan response adds another 31 bytes.
- Common types: Flags (`0x01`), service UUIDs (`0x02`/`0x03`), complete name (`0x09`), TX power (`0x0A`), manufacturer data (`0xFF`).
- Real example from a sensor: `02 01 06 | 0D 09 54 43 49 6F 54 2D 53 65 6E 73 6F 72`, i.e. Flags = `0x06` (discoverable, no BR/EDR) and name "TCIoT-Sensor".
- **Why do only some devices show a name?** The name may be only in the scan response, which only reaches *active* scanners.
- 31 bytes cannot hold everything: the developer chooses between name, UUID or data, and defers the rest to the scan response or to GATT.

Beacons and Distance Estimation (2/2)

- A **beacon** is a non-connectable advertiser broadcasting an identifier: *iBeacon* (Apple: UUID, major, minor, power at 1 m) and *Eddystone* (Google, service UUID `0xFEAA`).
- The receiver measures the **RSSI** and estimates distance with the log-distance model: $d = 10^{(P_{1\,m} - \mathrm{RSSI})/(10\,n)}$, with n between 2 (free space) and 4 (indoors).
- Example: $P_{1\,m} = -59$ dBm, RSSI $= -75$ dBm, $n = 2.5 \Rightarrow d \approx 4.4$ m.
- RSSI varies by several dB because of reflections and the human body: it supports zones (near, far), not exact metres.
- 5.1 (AoA/AoD) and 6.0 (*Channel Sounding*) exist precisely to overcome this limit.

Connection Establishment and Parameters (1/2)

- A scanning device becomes an initiator when it sends the advertiser a connection request packet, carrying the set of link layer parameters intended for the future slave.
- **Channel Map:** indicates which data channels are used during the connection.
- **Hop Increment:** a random value between 5 and 16, used by the channel selection algorithm.
- **Connection Interval:** a multiple of 1.25 ms, between 7.5 ms and 4.0 s.

Connection Establishment and Parameters (2/2)

- **Supervision Timeout:** a multiple of 10 ms, between 100 ms and 32.0 s.
- **Slave Latency:** a value between 0 and 499.

Connection Events and Slave Latency (1/2)

- Connection events occur periodically, with the period defined by the *connection interval* parameter.
- Each event takes place on a data channel, between 0 and 36, the next channel being determined by the *hop increment* parameter.
- Events occur even when neither party has data to send, which keeps the connection alive but consumes energy.
- *Slave latency* defines the maximum number of connection events the slave may skip.

Connection Events and Slave Latency (2/2)

- This mechanism allows the slave to skip events when it has no data to transmit, reducing consumption without terminating the connection.
- If the slave does not respond during an event, the master retransmits the packet in subsequent events until it obtains a reply.

Connection Parameters and Their Effect



Trade-offs in Parameter Selection (1/2)

- **Short connection interval:** higher consumption, higher throughput and shorter wait before data is sent.
- **Long connection interval:** reduced consumption, lower throughput and longer wait before data is sent.
- **Zero or low slave latency:** higher consumption, with more immediate data reception.
- **High slave latency:** reduced consumption, with less immediate data reception.

Trade-offs in Parameter Selection (2/2)

- If the slave finds the parameters unsuitable, it may send the master a connection update request, proposing an interval within a desired range.
- These parameters are, in practice, the main instrument available to the developer for balancing battery life against responsiveness.

Energy Budget: Order of Magnitude

- Average current over one cycle: $I_{avg} = \dfrac{I_{ev}\, t_{ev} + I_{sleep}\,(T - t_{ev})}{T}$, where T is the advertising or connection interval.
- Example: advertising every 1 s, 1 ms event at 8 mA, sleep at 2 µA $\Rightarrow$ $I_{avg} \approx 10$ µA.
- CR2032 cell (≈ 220 mAh): $220\,\text{mAh} / 0.010\,\text{mA} \approx 22\,000$ h, about 2.5 years (less in practice: self-discharge and current peaks).
- Moving the interval to 100 ms multiplies the average current by about 10; in return discovery becomes almost immediate.
- Classic Bluetooth, with a permanent link in the tens of mA, cannot follow this operating profile.

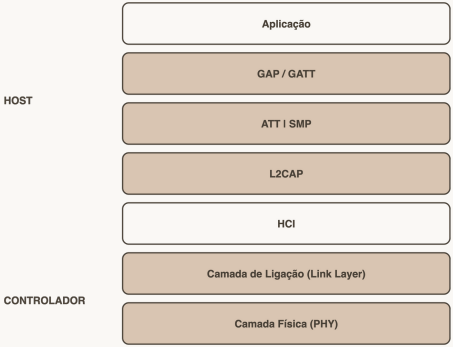
Connection Termination (1/2)

- The connection may be terminated voluntarily by either party, master or slave, for any reason.
- It may also end through expiry of the supervision timer.
- When the timer reaches the *supervision timeout* value, the device considers the connection lost.

Connection Termination (2/2)

- In that case it leaves the connection state and returns to advertising, scanning or standby, depending on its role.
- This mechanism ensures that a device does not wait indefinitely for a peer that has stopped responding.

The BLE Protocol Stack



O HCI separa o host (tipicamente software) do controlador (rádio)

Protocol Stack: HCI and L2CAP (1/2)

- The BLE stack is divided into two main sections: the *controller*, which handles the radio and the link layer, and the *host*, which supports the upper protocols.
- The **Host Controller Interface (HCI)** is the boundary between the two: it performs no processing of its own, serving as the interface through which the host communicates with the lower layers.

Protocol Stack: HCI and L2CAP (2/2)

- The **Logical Link Control and Adaptation Protocol (L2CAP)** allows higher-level protocols and applications to transmit and receive data packets of limited size (the default ATT MTU is 23 bytes, detailed later).
- L2CAP also provides channel management, allowing logical channels between two endpoints, supported by the link layer.

MTU, Useful Throughput and UUIDs

- The default **ATT_MTU** is **23 bytes**; minus 3 bytes of ATT header, each read or notification carries 20 useful bytes. This is the 23-byte figure to keep in mind in applications.
- The MTU is negotiated after connecting (up to 517 bytes) and, since 4.2, the link-layer packet can reach 251 bytes (*Data Length Extension*).
- 16-bit **UUIDs** are shorthand for the SIG base UUID: `0x180F` ≡ `0000180F-0000-1000-8000-00805F9B34FB`. This long form is what `gatttool` prints.
- Multi-byte values are *little-endian*: `2E 09` = `0x092E` = 2350, i.e. 23.50 °C in hundredths.

Network Topology (1/2)

- BLE is organised in a star topology: the master device manages the connection and may be connected to several slaves.
- Each slave may only be connected to one master at a time within the same connection.
- Each slave communicates with the master on a distinct physical channel.
- A device may maintain physical connections with more than one master simultaneously, and may even act as master and slave at the same time.

Network Topology (2/2)

- Hence the terms *piconet*, for the set formed by one master and its slaves, and *scatternet*, for the interconnection of several piconets.
- In a connected BLE network, two peripherals do not communicate directly: all messages pass through the central device.

Generic Access Profile (GAP)

- It controls connection and advertising processes, and is what makes a device visible to others.
- It determines how two devices may, or may not, interact with each other, and defines the role of each.
- It defines a base profile that all Bluetooth devices implement, articulating the various layers to form the minimum requirements of a Bluetooth device.
- It further defines generic procedures for connection-related services: device discovery, connection establishment, management and termination, and initiation of security features.

GAP: Roles and Discovery Modes (1/2)

- **Broadcaster:** a non-connectable advertiser, which only broadcasts information.
- **Observer:** it looks for advertisements but cannot initiate connections.
- **Peripheral:** a connectable advertiser, operating as slave in a single link layer connection.
- **Central:** it looks for advertisements and initiates connections, operating as master in one or several connections.
- **Non-discoverable mode:** the device emits no advertisements.

GAP: Roles and Discovery Modes (2/2)

- **Limited discoverable mode:** the device advertises for a limited period, after which it returns to standby.
- **General discoverable mode:** the device advertises continuously.
- GAP also manages the data sent in advertising packets and in scan request responses.

Security: Security Manager Protocol (1/2)

- The Security Manager Protocol (SMP) performs authentication and key management, using AES-128 as the encryption algorithm.
- It works together with GAP to manage relationships between devices, through three distinct mechanisms.
- **Pairing:** establishes encryption between two devices after the connection has been created.

Security: Security Manager Protocol (2/2)

- **Authentication:** verifies that the peer device is trusted, protecting against man-in-the-middle attacks.
- **Bonding:** establishes a long-term relationship between devices, storing security and identity information.

Pairing: Keys and Capabilities (1/2)

- Pairing may be initiated by the central or by the peripheral device.
- The two devices generate and exchange short-term keys, used to decrypt the data packets.
- Either device may request bonding to be enabled, creating a long-term relationship.

Pairing: Keys and Capabilities (2/2)

- In that case a long-term key is generated, exchanged and stored, which allows the connection to be encrypted again quickly on reconnection, without repeating the whole pairing process.
- Each device declares its input and output capabilities: **DisplayOnly**, with no input but with data output; **DisplayYesNo**, with input limited to yes or no and the ability to display data; **KeyboardOnly**, with passkey entry but no display; **NoInputNoOutput**, with no user interaction at all; and **KeyboardDisplay**, with both capabilities.

Pairing Methods (1/2)

- The combination of the input and output capabilities of the two devices determines which pairing method is used.
- **Passkey entry:** one device displays a randomly generated key, and the other requires the user to enter it. This method produces an authenticated connection, with protection against man-in-the-middle attacks.

Pairing Methods (2/2)

- **“Just Works”:** the process completes without entering any key. The connection is encrypted but not authenticated, which leaves it vulnerable to man-in-the-middle attacks.
- If one of the devices has neither input nor output capability, “Just Works” is the only possible method, a limitation with significant security implications for simple sensors.

Security: Evolution and Vulnerabilities

Mechanism	How it works	Known weakness
LE Legacy (4.0/4.1)	temporary key (TK) exchanged; *Just Works*: TK = 0	a passive sniffer recovers the long-term key (Crackle tool)
LE Secure Connections (4.2)	ECDH on P-256; numeric comparison	*Just Works* still unauthenticated against a man in the middle
Classic SSP (2.1)	ECDH, numeric comparison	KNOB (2019) reduces key entropy; BIAS (2020) allows impersonation
Stacks and SoCs	—	BlueBorne (2017), SweynTooth (2020): implementation flaws

– Good practice: require LE Secure Connections, authenticate sensitive reads, and use resolvable private addresses.

Attribute Protocol (ATT) (1/2)

- An attribute is a discrete value with three associated properties: an identifier (*handle*), which acts as an address; a type; and a set of permissions.
- ATT defines the protocol used over the air to read, write and discover attributes.
- The architecture is client-server: servers hold the data and expose it as attributes; clients wish to use it.

Attribute Protocol (ATT) (2/2)

- A device's client or server role is independent of its central or peripheral role in GAP, and of its master or slave role at the link layer.
- In an attribute table, the *handle* gives the attribute address, the type indicates what the data represents (through a UUID assigned by the Bluetooth SIG or a custom type) and the permissions determine whether and how the client may access the value.

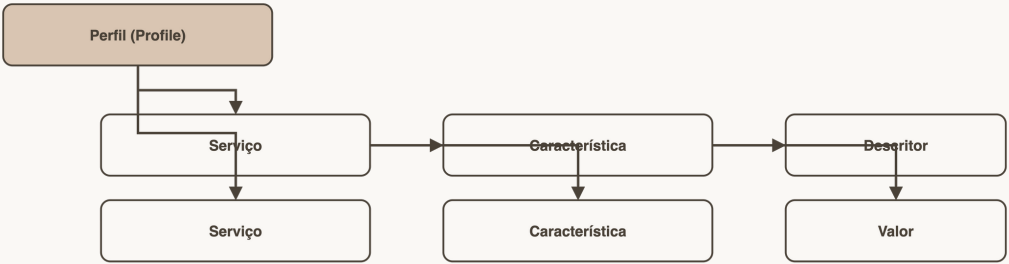
Generic Attribute Profile (GATT) (1/2)

- It defines how two BLE devices exchange data, through the concepts of service and characteristic.
- It only comes into operation once a dedicated connection has been established between the two devices, and communication is bidirectional.
- It establishes a client-server relationship: the peripheral is the GATT server, holding the service and characteristic definitions; the central is the GATT client, sending requests to the server.
- **Profile:** a predefined collection of services, compiled by the Bluetooth SIG or by the peripheral designer.

Generic Attribute Profile (GATT) (2/2)

- **Service:** a collection of characteristics, identified by a unique UUID of 16 to 128 bits.
- **Characteristic:** it encapsulates a single data point. Since it is possible to write to a characteristic, this is also the mechanism for sending data back to the peripheral.

GATT Attribute Hierarchy



Cada atributo tem um identificador (UUID), um handle e permissões de acesso

GATT: Transactions and Hierarchy (1/2)

- All transactions are initiated by the GATT client, which receives the corresponding response from the server.
- On establishing the connection, the peripheral suggests a connection interval; the central device attempts to contact it at each interval to check for new data.
- A profile is composed of one or more services required to satisfy a specific use case.
- A service may contain attributes designated characteristic values, which are the values used by the service.

GATT: Transactions and Hierarchy (2/2)

- Each characteristic value is necessarily preceded by a characteristic declaration attribute, describing its properties.
- Characteristics may further include optional descriptor attributes, with additional information about their content or configuration.

GATT: Client Procedures (1/2)

- GATT defines the procedures for using the attribute protocol to discover, read, write and obtain indications about attributes.
- It also defines the grouping and relationship between characteristics within a service or profile, and the procedures for configuring attribute broadcasting.
- **Discover characteristic by UUID:** searches the server for all attributes whose type matches the given UUID.
- **Read characteristic value:** reads the characteristic value at the specified *handle*.

GATT: Client Procedures (2/2)

- **Write characteristic value:** writes a new value to the given characteristic.
- These procedures form the practical basis of BLE programming, taken up again in the next lesson.

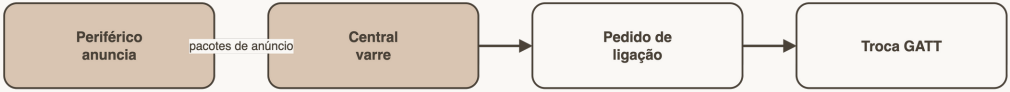
Notifications and Indications (CCCD) (1/2)

- Instead of the client repeating reads (*polling*), the server can **notify** when the value changes, saving energy and airtime.
- Subscription is done by writing the **CCCD** descriptor (UUID `0x2902`) of the characteristic: `0x0001` enables notifications, `0x0002` indications, `0x0000` disables.
- **Notification:** unacknowledged. **Indication:** the client confirms each message (more reliable, slower).
- In `gatttool`: `char-write-req <CCCD handle> 0100`, then lines `Notification handle = ... value: ....`
- The CCCD exists only in characteristics with the *notify* or *indicate* property; its handle follows the value handle.

Example: Battery Service (2/2)

- Service `0x180F`; Battery Level characteristic `0x2A19`: one byte, 0 to 100 (%).
- Read: `char-read-hnd 0x0008` $\Rightarrow$ 57; in decimal, $0\text{x}57 = 87$, i.e. 87 % charge.
- Common standard services: Generic Access `0x1800`, Device Information `0x180A`, Heart Rate `0x180D`, Environmental Sensing `0x181A`.
- A random 128-bit UUID indicates a vendor service: the meaning of the bytes is known only from documentation, or by reverse engineering.
- Rule of thumb: always interpret handle $\rightarrow$ UUID $\rightarrow$ properties $\rightarrow$ value format.

Advertising Broadcast and Connection-Based Exchange



Difusão sem ligação (beacon) termina no primeiro passo; a troca de dados estruturados exige ligação

Advertising-Based and Connection-Based Communication (1/2)

- BLE supports two distinct communication paradigms, and the choice between them shapes the whole application architecture.
- **Advertising-based communication:** connectionless, broadcast in nature, unidirectional. The peripheral emits data that any device in the vicinity may receive.
- **Connection-based communication:** connection-oriented, bidirectional, supported by GATT. It requires a dedicated connection to be established beforehand.

Advertising-Based and Connection-Based Communication

(2/2)

- Broadcasting avoids connection negotiation and is more energy-efficient, but offers neither delivery confirmation nor confidentiality.
- Connection-based communication allows reading, writing and notifications, at the cost of keeping the connection active.

Typical Applications (1/2)

- BLE was designed for very low consumption and low duty cycles, which defines its field of application.
- **Objects we carry:** watches, health and sports accessories, body sensors.
- **Objects around us:** location tags, tags used as keys, home automation sensors and actuators.
- **Communication within systems:** office automation, industrial automation and in-vehicle communication.

Typical Applications (2/2)

- **Proximity profile:** defines behaviour when a device moves away from its peer, to the point where the connection drops or signal attenuation exceeds a predefined threshold, generating an immediate alert.
- **Link loss service:** uses the alert level characteristic to signal connection loss on the device.

BLE Addresses and Privacy

Type	Characteristic
Public	assigned by the IEEE, fixed: trackable forever
Random static	generated at boot, fixed until restart
Resolvable private (RPA)	changes every $\approx$ 15 min; only a holder of the IRK key (after bonding) recognises it
Non-resolvable private	random and temporary; for beacons

- Almost all phones advertise with an RPA: so the address seen in a `lescan` changes between scans and is a **random** address.
- Practical consequence: connecting with `gatttool` requires the `-t random` option.
- If the tracker of the case study used an RPA, tracking by advertisements would stop being trivial; implementation flaws still allow it in some products.

Case Study: Activity Trackers (1/2)

- Activity trackers connect to the phone only when the corresponding application is running in the foreground.
- While the application runs, the tracker communicates actively and synchronises activity data, such as step count and calories.
- When the application stops running, the connection is terminated and the tracker returns to the advertising state.
- In this state, the device continues to emit advertising packets, which any receiver in the vicinity may capture.

Case Study: Activity Trackers (2/2)

- This results in a privacy leak: continuous advertising allows the presence and movement of the user to be tracked, even with no connection established.
- This example illustrates that design decisions at protocol level have direct consequences for privacy.

Management Tools on Linux (1/2)

- **hciconfig:** lists the BLE adapters attached to the system. With `hciconfig hciX up` and `hciconfig hciX down` the given adapter is enabled and disabled.
- **hcitool:** configures Bluetooth connections and sends specific commands to devices. The command `hcitool lescan` searches for BLE devices in the vicinity.
- **gatttool:** allows characteristics to be discovered, read and written. Run with the `-I` option, it opens an interactive mode where commands such as `connect <address>` may be issued.

Management Tools on Linux (2/2)

- These tools will be used in the laboratory sessions, when the Raspberry Pi takes on the role of BLE client.
- On the mobile application side, nRF Connect and BLE Scanner allow services and characteristics of nearby devices to be inspected.

Modern BlueZ Tools

Classic (deprecated)	Current equivalent
`hciconfig`	`bluetoothctl list`, `show`; `btmgmt info`
`hcitool lescan`	`bluetoothctl scan on` (shows RSSI, TxPower, advertisement data)
`gatttool -I`	`bluetoothctl`: `connect`, `menu gatt`, `list-attributes`, `select-attribute`, `read`, `notify on`
`hcidump`	`btmon` (live HCI trace and file recording)

- BlueZ marks `hciconfig`, `hcitool`, `gatttool`, `hcidump`, `rfcomm` and `sdptool` as deprecated. Raspberry Pi OS Bookworm (BlueZ 5.66) still ships them, but other distributions may not.
- In Python, the `bleak` library runs on Linux, macOS and Windows and is the natural route for the code in Lessons 05 and 06.

No Hardware: Emulation, Simulation and Offline Analysis

Option	Purpose
`ble_sim.py` (lab guide)	console simulator that mimics `hciconfig`, `hcitool lescan` and `gatttool`; no dependencies
Wireshark + `btsnoop` file	offline analysis of advertisements, ATT and GATT (filters `btle`, `btatt`); same file as the Android *HCI snoop log*
Bumble (Python)	Bluetooth stack with virtual controllers: two virtual devices talk without a radio (`pip install bumble`)
nRF Connect (Android/iOS)	the phone emulates a peripheral with configurable GATT, useful even without a Raspberry Pi

– None of this replaces the real radio (interference, RSSI, range), but it lets students practise reasoning about services, characteristics, notifications and advertising.

Traffic Capture for Analysis

- Android provides, from version 4.4 onwards, an option to record all Bluetooth packets entering and leaving the device.
- It is enabled in two steps: enable developer options in the system settings and, within those, enable the *Bluetooth HCI snoop log*.
- The resulting file can then be analysed with protocol analysis tools, allowing the real sequence of advertisements, connections and GATT transactions to be observed.
- This capability is useful both in development, to diagnose communication problems, and in the security analysis of BLE devices.

Lesson Summary (1/2)

- Classic Bluetooth and BLE share the brand and the operating band, but they are distinct and non-interoperable technologies: BLE trades throughput and message size for much lower consumption and latency.
- The link layer organises all communication around six states and two modes: advertising, for broadcasting, and connection, for bidirectional exchange.
- The connection parameters (interval, slave latency and supervision timer) are the main instrument for balancing battery life against responsiveness.

Lesson Summary (2/2)

- GAP defines visibility and roles; ATT defines the format of the exposed data; GATT defines how client and server exchange it through services and characteristics.
- Security rests on pairing, authentication and bonding, but the limited input and output capabilities of sensors push many implementations towards the "Just Works" method, with no protection against intermediaries.
- The next lesson applies these concepts in implementing a BLE server on Arduino.

Sources

Gonçalves, P. *Apresent4 — Bluetooth*, TCIoT, previous editions. Basis for the whole exposition on physical layer, link layer, GAP, GATT, pairing and Linux management tools.

Townsend, K. *et al.* (2014). *Getting Started with Bluetooth Low Energy*. O'Reilly.

Bluetooth SIG, *Core Specification* and *Assigned Numbers*; BlueZ and Bumble documentation (`google.github.io/bumble`).

Bluetooth SIG. *Bluetooth Core Specification*. Normative definition of GAP, GATT, ATT, L2CAP and SMP.

Collotta, M. *et al.* (2018). Bluetooth 5: A Concrete Step Forward toward the IoT. *IEEE Communications Magazine*. Available in `Leituras/`.

Jadhav, S. *et al.* (2016). Bluetooth 5 and Internet of Things: Potential and architecture. *SCOPES*. Available in `Leituras/`.

Thank you.

Questions and comments